



Π.Τ.Ε.Σ – ΕΡΓΑΣΙΑ 4

CERSTRUM via HOMOMORPHIC FILTERING
ΜΑΧΑΙΡΑΣ ΣΤΑΥΡΟΣ – ΑΕΜ:10046



ΜΑΘΗΜΑ: ΠΡΟΗΓΜΕΝΕΣ ΤΕΧΝΙΚΕΣ ΕΠΕΞΕΡΓΑΣΙΑΣ ΣΗΜΑΤΟΣ
ΤΜΗΜΑ ΗΛΕΚΤΡΟΛΟΓΩΝ ΜΗΧΑΝΙΚΩΝ & ΜΗΧΑΝΙΚΩΝ
ΥΠΟΛΟΓΙΣΤΩΝ
ΑΡΙΣΤΟΤΕΛΕΙΟ ΠΑΝΕΠΙΣΤΗΜΙΟ ΘΕΣΣΑΛΟΝΙΚΗΣ
ΑΚΑΔΗΜΑΪΚΟ ΕΤΟΣ 2022-2023
1 ΣΕΠΤΕΜΒΡΙΟΥ 2023

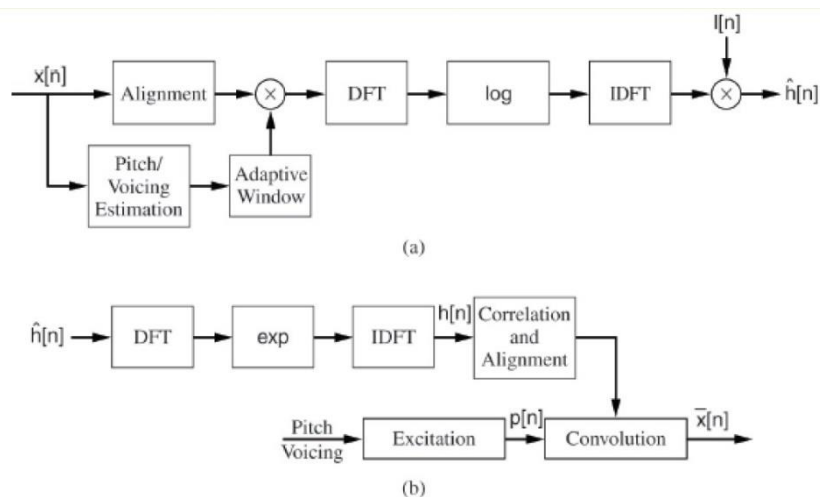
Πίνακας Περιεχομένων

Εργασία	2
Ανάλυση	3
Κώδικας <i>'MATLAB'</i>	16

Εργασία

1. Στο στάδιο αυτό καταγράφονται δέκα δείγματα φωνής από ένα αρσενικό και ένα θηλυκό άτομο για τους πέντε φωνητικούς ήχους – “a”, “e”, “i”, “o”, “u”. Πρέπει να ληφθούν σημαντικά υπόψη οι συνθήκες κάτω υπό τις οποίες έγινε η λήψη των φωνητικών δειγμάτων (είδος μικροφώνου, χρησιμοποιούμενο λογισμικό).
2. Να υπολογιστεί το *cepstrum* για κάθε φωνητικό δείγμα και να συζητηθούν οι διαφορές, ποσοτικά και ποιοτικά, ανάμεσα στην αρσενική και τη θηλυκή φωνή γενικά και ειδικότερα για κάθε ξεχωριστό φωνητικό ήχο. Οι γυναικείες φωνές, σε γενικές γραμμές, θα έχουν περισσότερες κορυφώσεις από τις αντρικές στο *cepstrum domain*. Να συζητηθεί το γιατί συμβαίνει αυτό.
3. Να γίνει φιλτράρισμα (*liftering*) των σημάτων στο *cepstrum domain*. Να σχεδιαστεί ένα παράθυρο (το μήκος του παραθύρου αποτελεί σημαντική σχεδιαστική παράμετρο και πρέπει να συζητηθεί το πως και το γιατί έχει επιλεγεί – μπορεί να είναι η ίδια ή και διαφορετική για κάθε δείγμα φωνής) για να εξαλειφθεί η επίδραση της συνάρτησης μεταφοράς. Στη συνέχεια να υπολογιστεί το σήμα στο πεδίο του χρόνου για το φιλτραρισμένο (*windowed*) αποτέλεσμα, για να προκύψει το αποσυνελιγμένο (*deconvolved*) σήμα. Υπάρχει καμία διαφορά μεταξύ του προκύπτοντος σήματος και του πραγματικού ηχογραφημένου σήματος στο πεδίο του χρόνου;
4. Να γίνει προσπάθεια σύνθεσης του αρχικού φωνητικού σήματος ακολουθώντας την διαδικασία της *Εικόνας 1*.

Εικόνα 1: Διαδικασία Σύνθεσης Αρχικού Σήματος



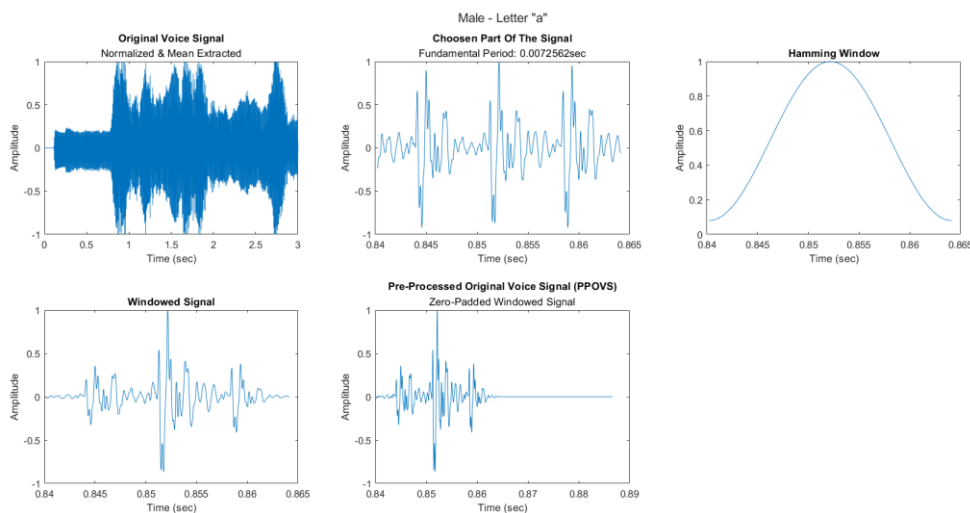
Ανάλυση

1^ο Στάδιο: Η ηχογράφηση των φωνητικών δειγμάτων έγινε μέσα από το περιβάλλον του 'MATLAB' και συγκεκριμένα με τον κώδικα της συνάρτησης 'recording.m'. Γενικότερα χρησιμοποιήθηκε το 'Record And Play Audio' toolbox του προγράμματος και συγκεκριμένα οι εντολές 'audiorecorder', 'recordblocking', 'getaudiodata'. Επιλέχθηκαν, ο ρυθμός δειγματοληψίας (Sampling Rate) στα 44100Hz, τα bits ανά δείγμα (Bits Per Sample) στα 16bit, η διάρκεια καταγραφής (Duration Time) στα 3sec και το κανάλι μονοφωνικό. Σαν μικρόφωνο χρησιμοποιήθηκαν ακουστικά 'hands free' και συγκεκριμένα το μοντέλο 'JBL TUNE205'.

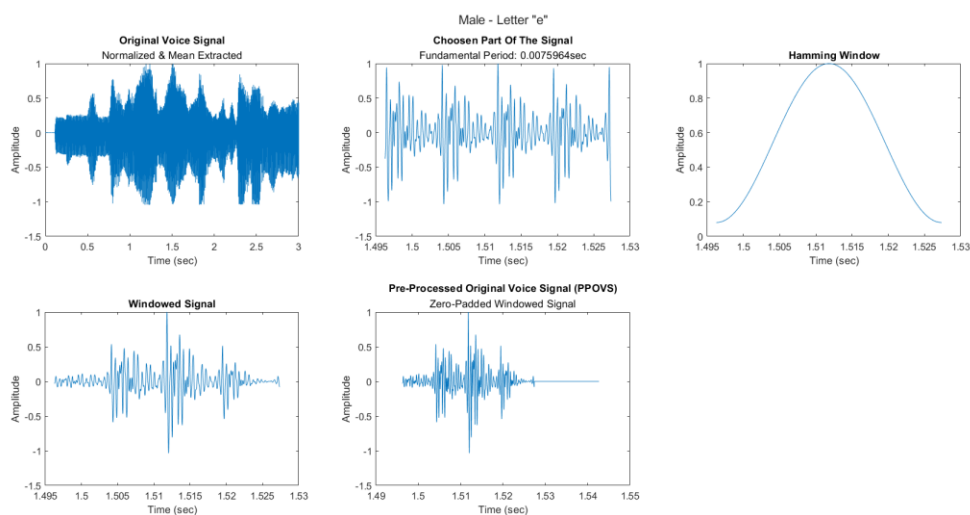
Στο σημείο αυτό παρουσιάζονται οι πρωτότυπες κυματομορφές των ηχητικών σημάτων καθώς και μία μικρή προ-επεξεργασία στην οποία υποβλήθηκαν ώστε να υπολογιστεί στο επόμενο στάδιο το *cepstrum*. Συγκεκριμένα, το κομμάτι της προ-επεξεργασίας περιλαμβάνει, αρχικά, την εξαγωγή της μέσης τιμής από το σήμα και την κανονικοποίηση του. Στη συνέχεια, εφαρμόζεται στο σήμα ένα παράθυρο τύπου *hamming*, μήκους ίσο με ένα πολλαπλάσιο της θεμελιώδους περιόδου του σήματος (το πολλαπλάσιο βρίσκεται στο διάστημα [3, 5]), με κέντρο τη μέγιστη κορυφή του ηχητικού σήματος. Ακολουθεί η προσθήκη των απαραίτητων μηδενικών (*zero padding*), ώστε το μήκος του προκύπτοντος σήματος να είναι δύναμη του δύο για να μπορεί να εφαρμοσθεί στην πορεία κατάλληλα ο μετασχηματισμός *fourier* (*fast fourier transform – fft*). Στο σημείο αυτό αξίζει να σημειωθεί ότι η θεωρία προβλέπει το προαναφερθέν πολλαπλάσιο να βρίσκεται στο διάστημα [2, 3]. Η τροποποίηση των τιμών αυτών προέκυψε μετά από το λεγόμενο 'trial & error', ώστε να υπάρχει καλύτερη απεικόνιση στο *cepstrum domain* στο επόμενο στάδιο της εργασίας.

Όλα τα παραπάνω παρουσιάζονται αναλυτικά στα Διαγράμματα 1 έως 5 για την ανδρική φωνή και στα Διαγράμματα 6 έως 10 για την γυναικεία φωνή.

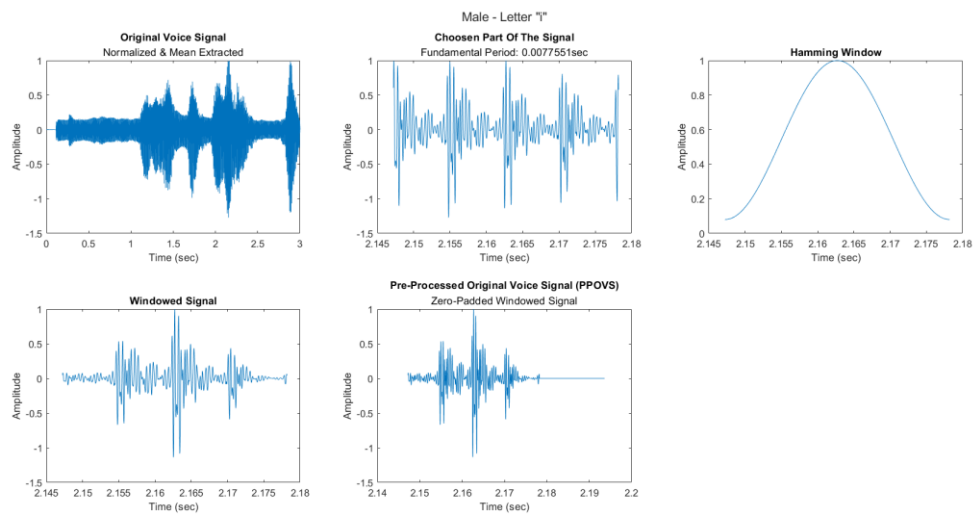
Διάγραμμα 1: Ανδρική Φωνή – Γράμμα “α” – Πρωτότυπο Σήμα & Προ-επεξεργασία



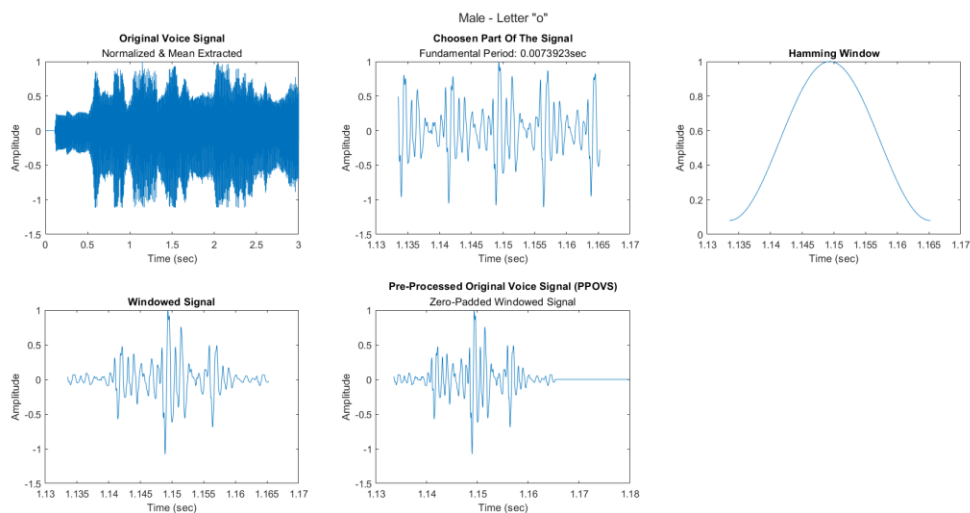
Διάγραμμα 2: Ανδρική Φωνή – Γράμμα “ε” – Πρωτότυπο Σήμα & Προ-επεξεργασία



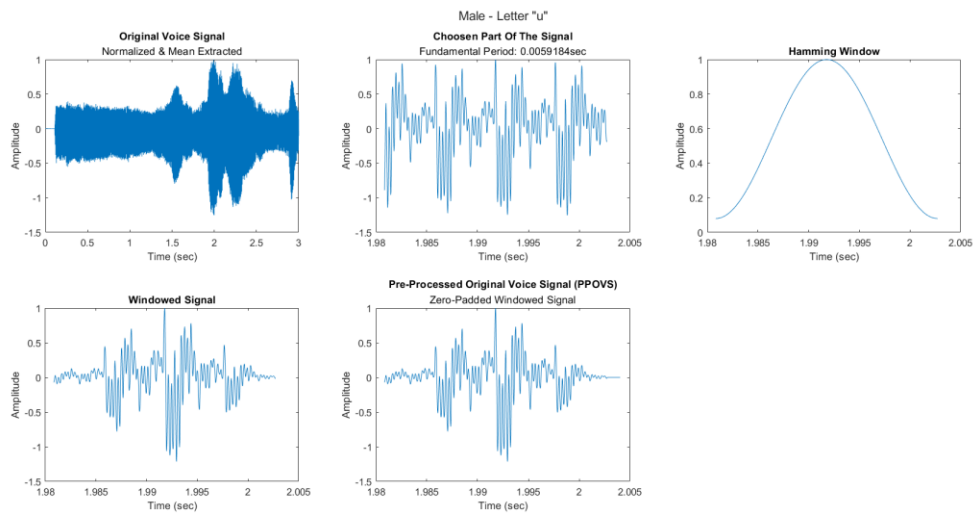
Διάγραμμα 3: Ανδρική Φωνή – Γράμμα “ι” – Πρωτότυπο Σήμα & Προ-επεξεργασία



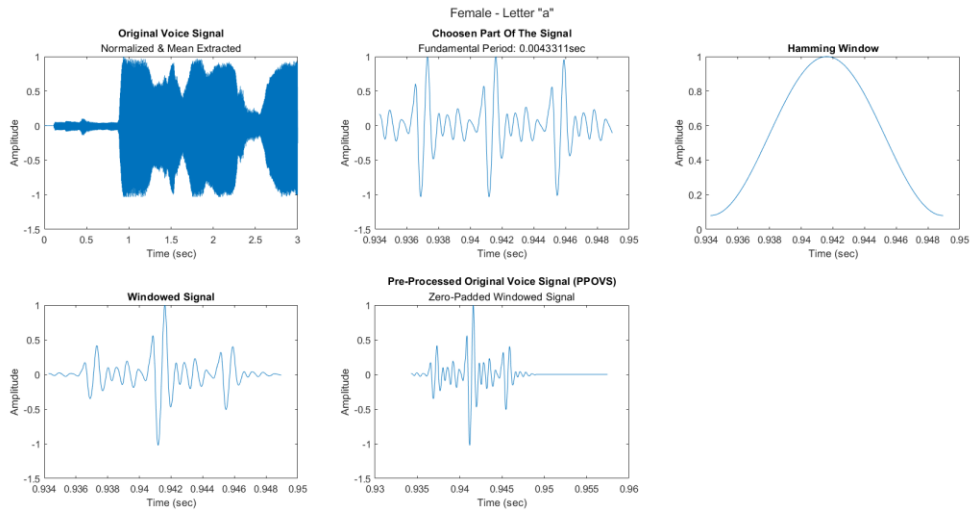
Διάγραμμα 4: Ανδρική Φωνή – Γράμμα “ο” – Πρωτότυπο Σήμα & Προ-επεξεργασία



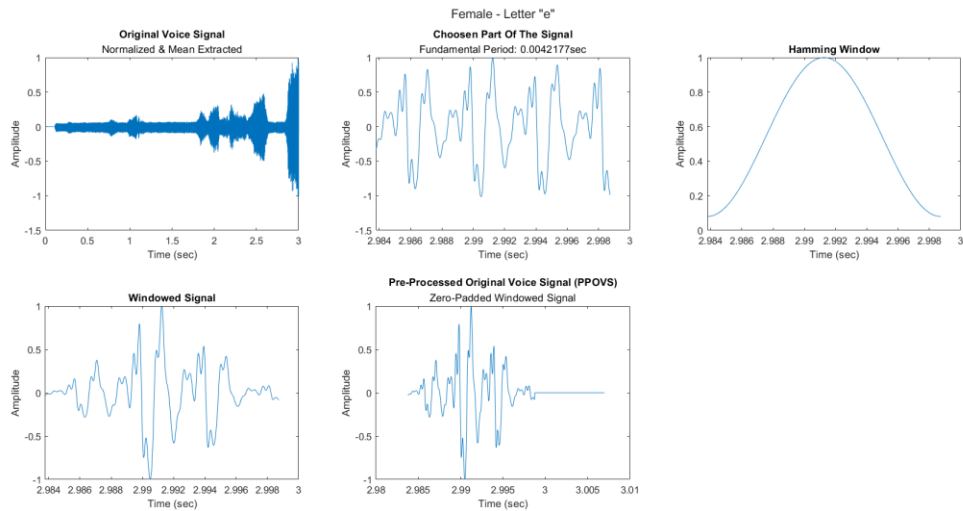
Διάγραμμα 5: Ανδρική Φωνή – Γράμμα “υ” – Πρωτότυπο Σήμα & Προ-επεξεργασία



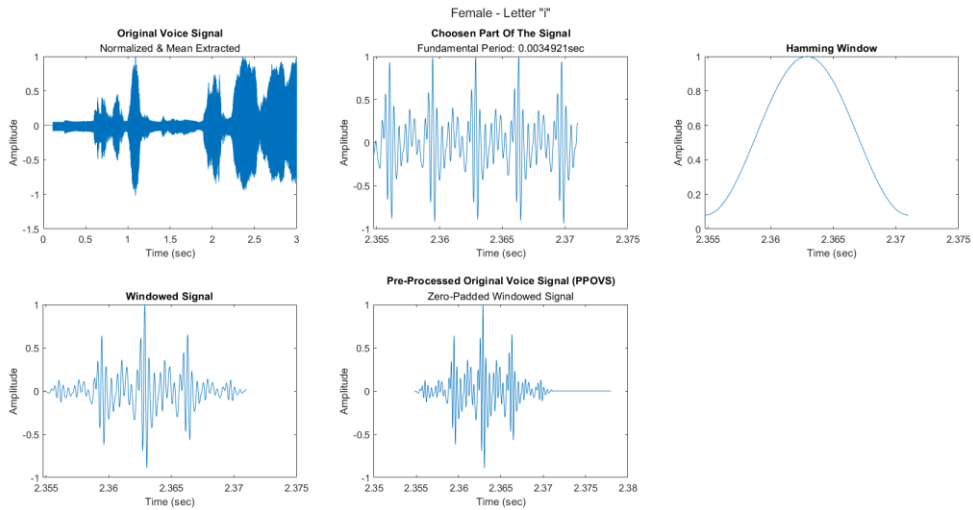
Διάγραμμα 6: Γυναικεία Φωνή – Γράμμα “α” – Πρωτότυπο Σήμα & Προ-επεξεργασία



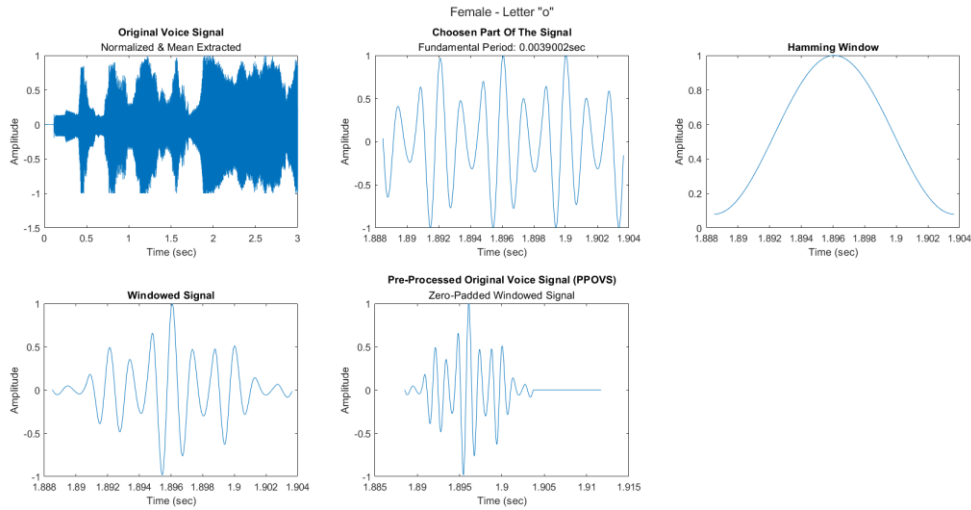
Διάγραμμα 7: Γυναικεία Φωνή – Γράμμα “ε” – Πρωτότυπο Σήμα & Προ-επεξεργασία



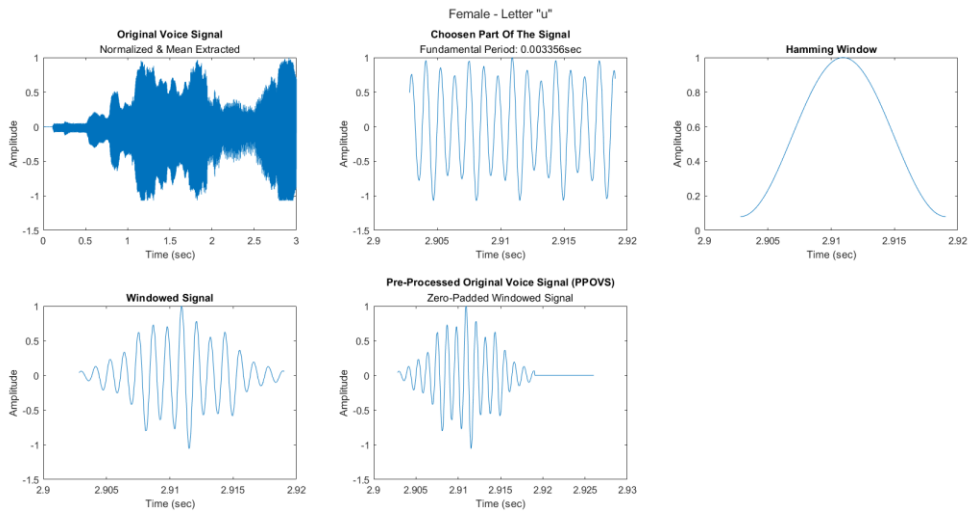
Διάγραμμα 8: Γυναικεία Φωνή – Γράμμα “ι” – Πρωτότυπο Σήμα & Προ-επεξεργασία



Διάγραμμα 9: Γυναικεία Φωνή – Γράμμα “ο” – Πρωτότυπο Σήμα & Προ-επεξεργασία



Διάγραμμα 10: Γυναικεία Φωνή – Γράμμα “υ” – Πρωτότυπο Σήμα & Προ-επεξεργασία



Παρακάτω, παρατίθενται οι τιμές της θεμελιώδους περιόδου, στον Πίνακα 1, καθώς και τα πολλαπλάσια που χρησιμοποιήθηκαν στην προ-επεξεργασία, στον Πίνακα 2, για κάθε φωνητικό δείγμα.

Πίνακας 1: Θεμελιώδης Περίοδος Ηχητικών Σημάτων (msec)

	“a”	“e”	“i”	“o”	“u”
Ανδρική Φωνή	73	76	78	74	59
Γυναικεία Φωνή	43	42	35	39	34

Πίνακας 2: Πολλαπλάσια Θεμελιώδους Περιόδου – Εφαρμογή Παραθύρου

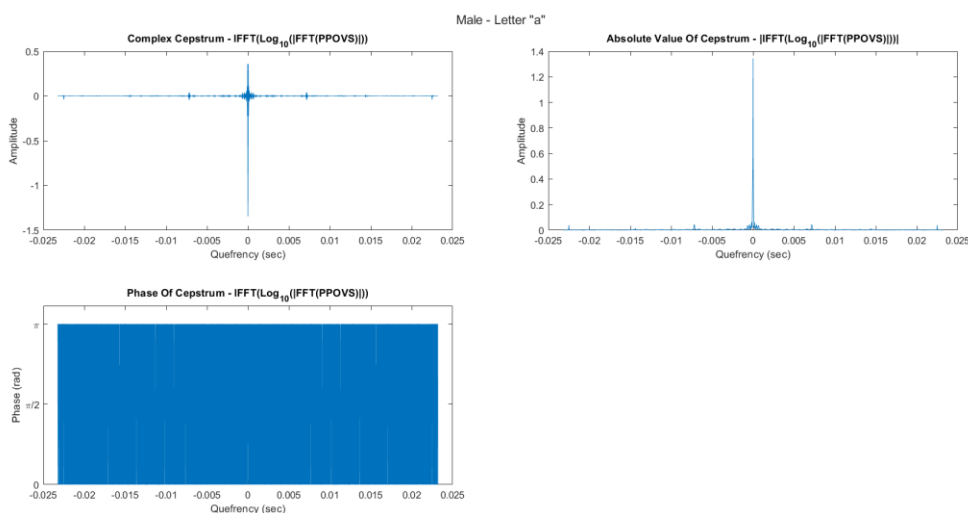
	“a”	“e”	“i”	“o”	“u”
Ανδρική Φωνή	3.3	4.1	4	4.3	3.7
Γυναικεία Φωνή	3.4	3.55	4.65	3.89	4.84

Γενικά, είναι αντιληπτό ότι η γυναικεία φωνή έχει μικρότερη θεμελιώδη περίοδο συγκριτικά με την ανδρική, γεγονός που ήταν σχετικά αναμενόμενο, καθώς οι ανδρικές φωνές τείνουν να είναι πιο μπάσες από τις γυναικείες. Αυτό σημαίνει πως κυμαίνονται σε μικρότερες συχνότητες και συνεπώς η περίοδος ταλάντωσης των φωνητικών χορδών είναι μεγαλύτερη.

2^ο Στάδιο: Το στάδιο αυτό ακολουθεί την προ-επεξεργασία του σήματος και υλοποιεί τον υπολογισμό του *cepstrum*. Η διαδικασία που ακολουθείται απαιτεί τον υπολογισμό του *fft* για το προ-επεξεργασμένο σήμα, την λογαριθμοποίηση του με χρήση του δεκαδικού λογάριθμου (*common logarithm*) και τον υπολογισμό του αντίστροφου μετασχηματισμού *fourier* (*inverse fourier transform – ifft*). Το σήμα που προκύπτει από αυτή την διαδικασία ονομάζεται *cepstrum* είναι συμμετρικό ως προς την αρχή των αξόνων.

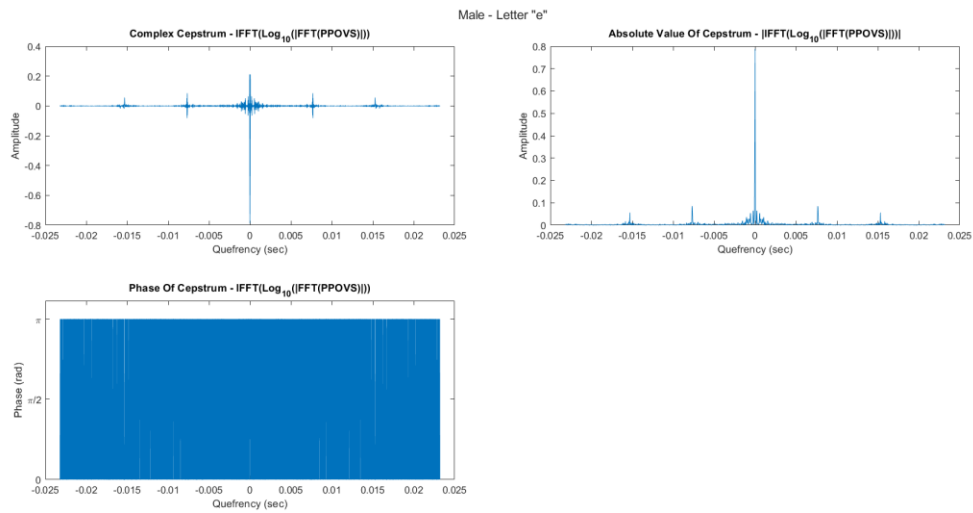
Ακολουθεί η παρουσίαση των αποτελεσμάτων για τον υπολογισμό του *cepstrum* στα Διαγράμματα 11 έως 15 για την ανδρική φωνή και στα Διαγράμματα 16 έως 20 για

Διάγραμμα 11: Ανδρική Φωνή – Γράμμα “a” – Cepstrum

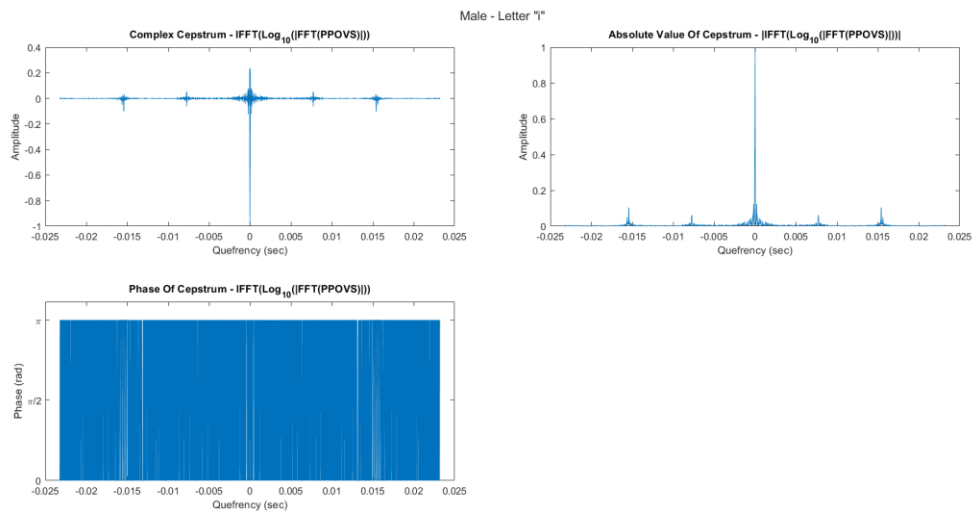


την γυναικεία φωνή. Στο αρχείο *'demo.m'* περιλαμβάνονται και οι απεικονίσεις για την ενδιάμεση διαδικασία με τον υπολογισμό του fft και την λογαριθμοποίηση.

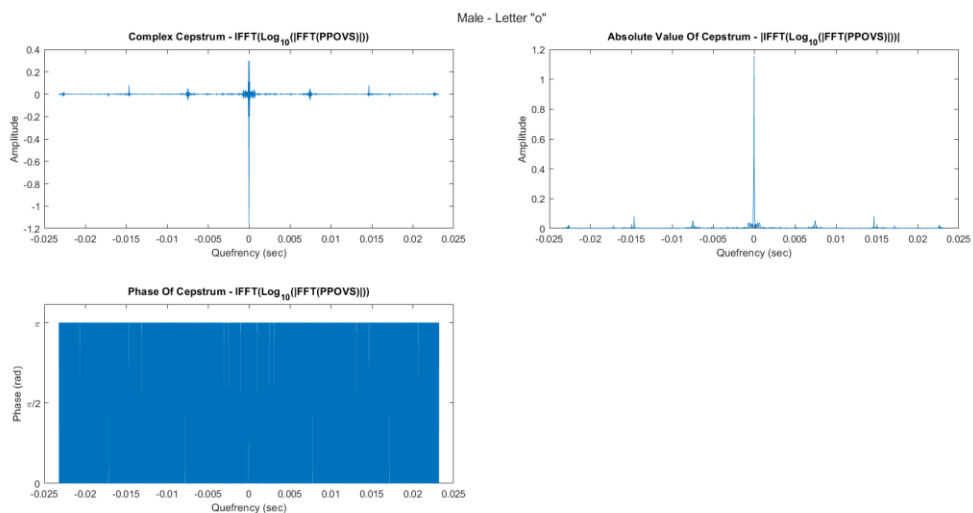
Διάγραμμα 12: Ανδρική Φωνή – Γράμμα “e” – Cepstrum



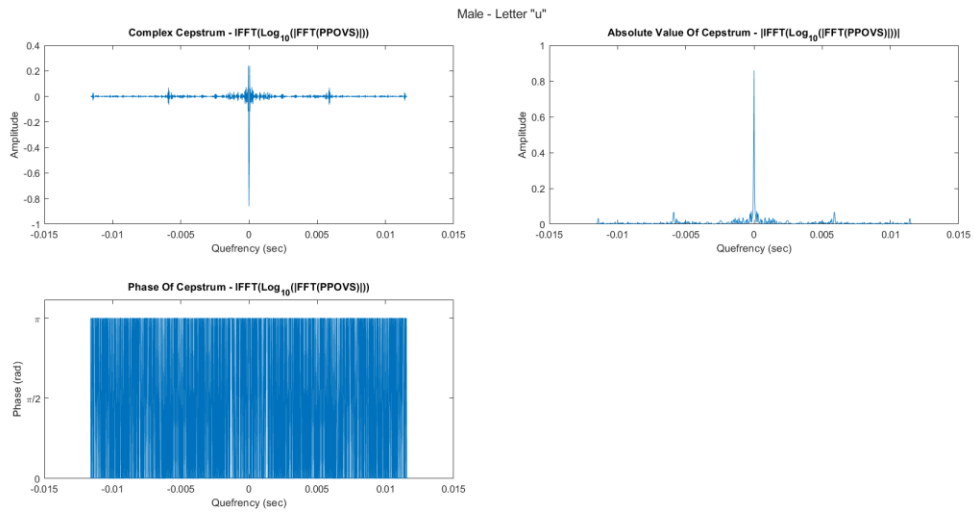
Διάγραμμα 13: Ανδρική Φωνή – Γράμμα “i” – Cepstrum



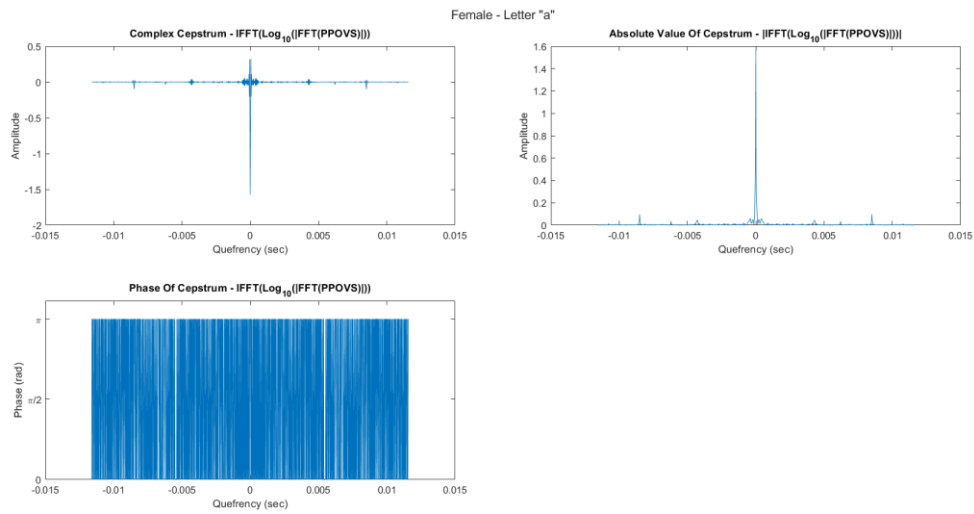
Διάγραμμα 14: Ανδρική Φωνή – Γράμμα “o” – Cepstrum



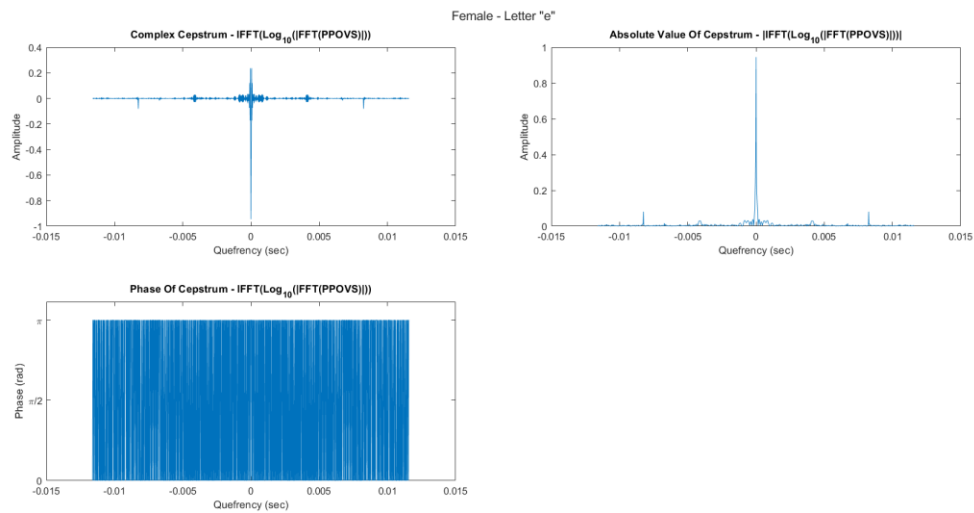
Διάγραμμα 15: Ανδρική Φωνή – Γράμμα “υ” – Cepstrum



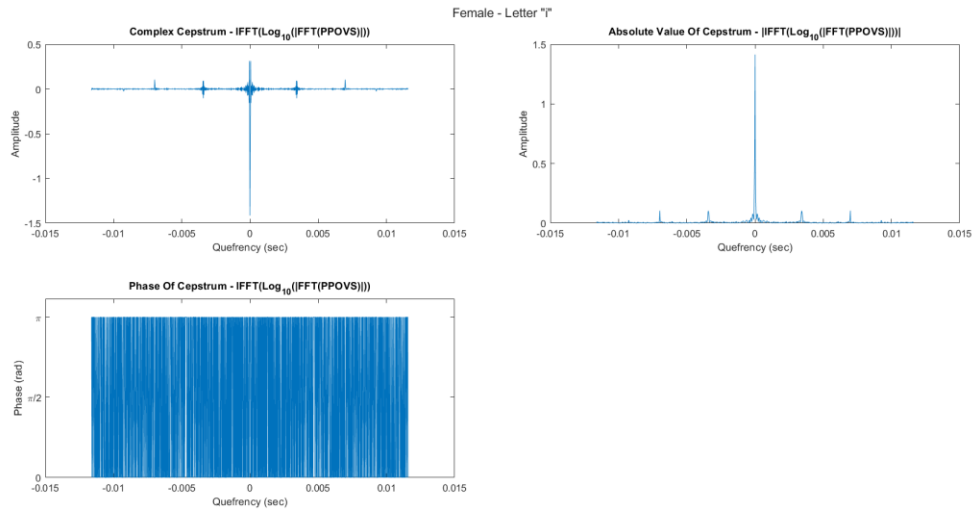
Διάγραμμα 16: Γυναικεία Φωνή – Γράμμα “α” – Cepstrum



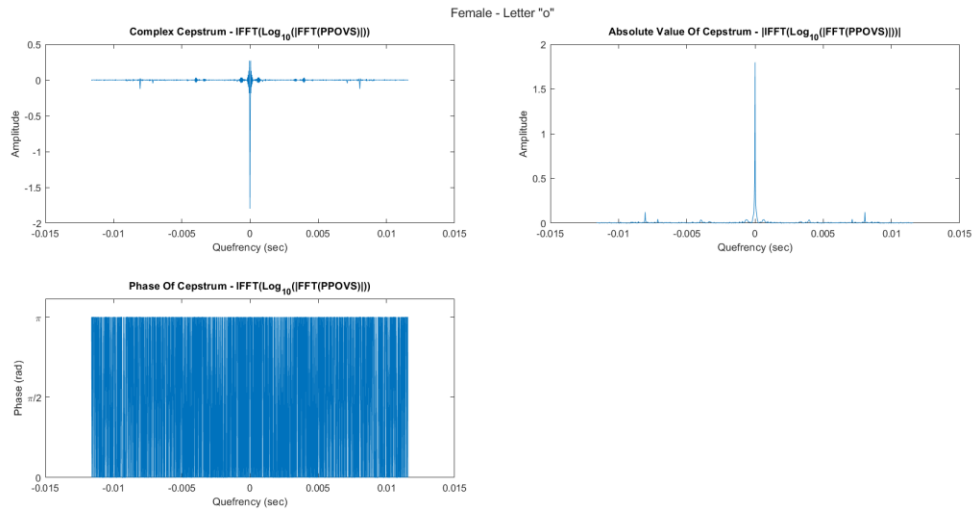
Διάγραμμα 17: Γυναικεία Φωνή – Γράμμα “ε” – Cepstrum



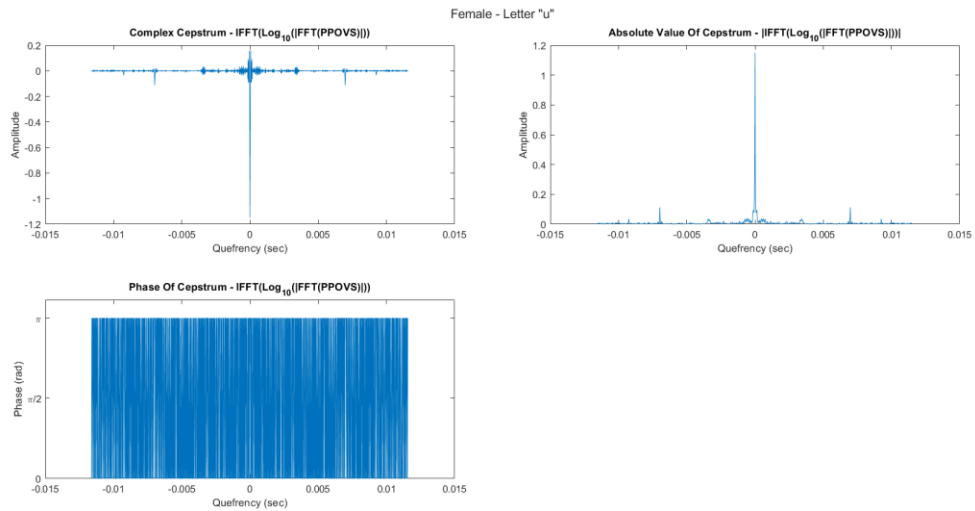
Διάγραμμα 18: Γυναικεία Φωνή – Γράμμα “i” – Cepstrum



Διάγραμμα 19: Γυναικεία Φωνή – Γράμμα “o” – Cepstrum



Διάγραμμα 20: Γυναικεία Φωνή – Γράμμα “u” – Cepstrum

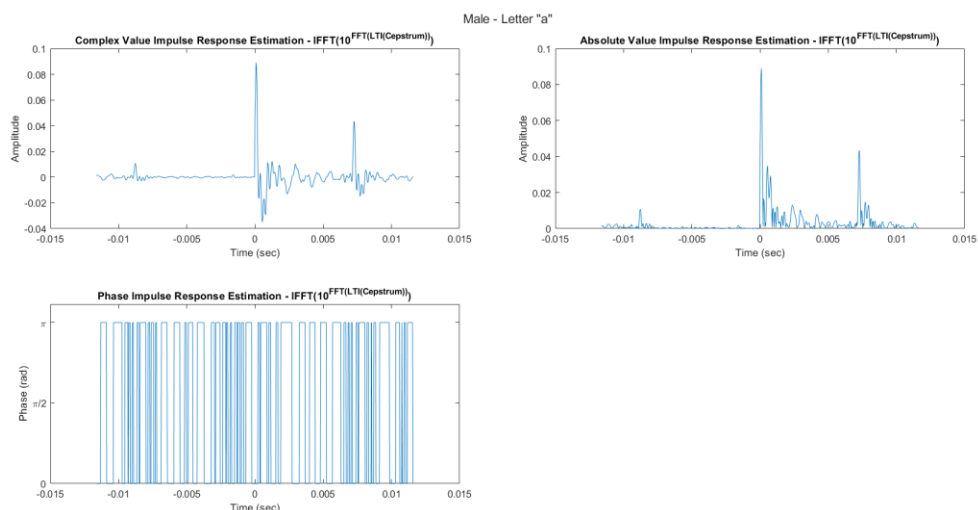


Σε γενικές γραμμές, το *cepstrum* φαίνεται να έχει παρόμοια απεικόνιση όσον αφορά και τα διαφορετικά φωνήεντα αλλά και το διαφορετικό φύλο. Στις περισσότερες περιπτώσεις γίνονται αντιληπτές δύο με τρεις αρμονικές (*rahnionics*) περίπου σε ακέραια πολλαπλάσια της θεμελιώδους συχνότητας του σήματος. Παρατηρείται, μάλιστα, ότι όσο πιο μακριά βρίσκεται η αρμονική από την αρχή των αξόνων τόσο μεγαλύτερο πλάτος έχει. Βέβαια, από τη στιγμή που υπάρχουν, ως επί το πλείστον, δύο αρμονικές σε κάθε διάγραμμα, είναι πιο δόκιμο να σημειωθεί ότι το πλάτος της δεύτερης αρμονικής είναι μεγαλύτερο από το εκείνο της πρώτης. Συγκρίνοντας τα ανδρικά δείγματα με τα γυναικεία παρατηρείται πως οι αρμονικές παρουσιάζονται πιο “πυκνές” και πιο “ευδιάκριτες” στα πρώτα.

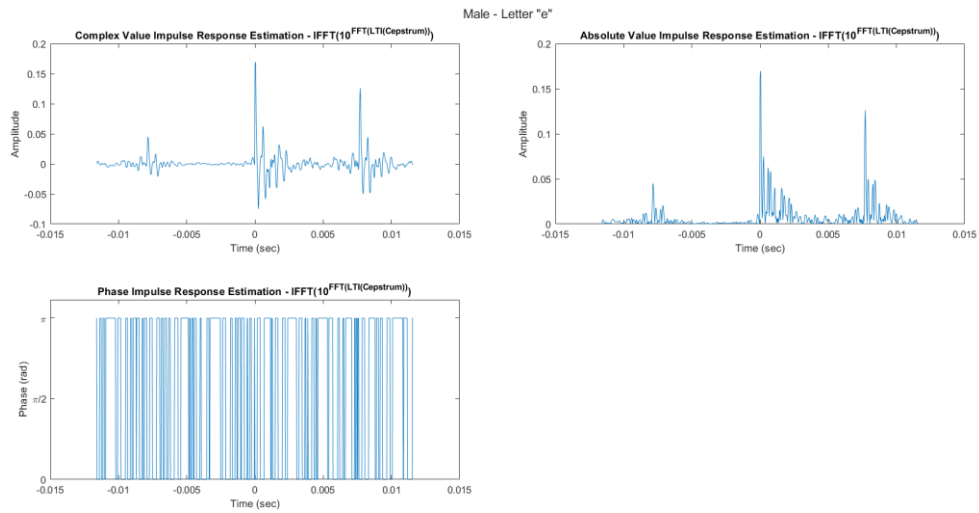
3^ο Στάδιο: Στο στάδιο αυτό γίνεται χαμηλοπερατό φιλτράρισμα του *cepstrum* (*low pass liftering*) με χρήση ενός τετραγωνικού παραθύρου. Το μήκος του παραθύρου έχει ορισθεί, για κάθε φωνητικό δείγμα, ίσο με το τριπλάσιο της θεμελιώδους περιόδου του σήματος και εφαρμόζεται με κέντρο την αρχή των αξόνων (0, 0). Στη συνέχεια, υπολογίζεται ο *fft* του μισού σήματος (για θετικό χρόνο, συμπεριλαμβάνονται στον υπολογισμό και τα μηδενικά που προέκυψαν από το φιλτράρισμα), γίνεται απολογαριθμοποίηση, υψώνοντας το προκύπτον σήμα στη δύναμη του δέκα και τελικά υπολογίζεται ο *ifft*. Με την μέθοδο αυτή γίνεται η εκτίμηση της κρουστικής απόκρισης (*impulse response*) του συστήματος (με τον όρο σύστημα χαρακτηρίζεται η επίδραση του ανθρώπινου οργανισμού από την στιγμή που παράγεται η φωνή από τις φωνητικές χορδές έως και τη στιγμή που φτάνει στην έξοδο).

Παρακάτω, ακολουθεί η εκτίμηση της κρουστικής απόκρισης του συστήματος για την ανδρική φωνή στα Διαγράμματα 21 έως 25 και για την γυναικεία φωνή στα Διαγράμματα 26 έως 30 αντίστοιχα. Και για το στάδιο αυτό περιλαμβάνονται στο αρχείο ‘*demo.m*’ οι απεικονίσεις για την ενδιάμεση διαδικασία με την εφαρμογή του παραθύρου, τον υπολογισμό του *fft* και την απολογαριθμοποίηση.

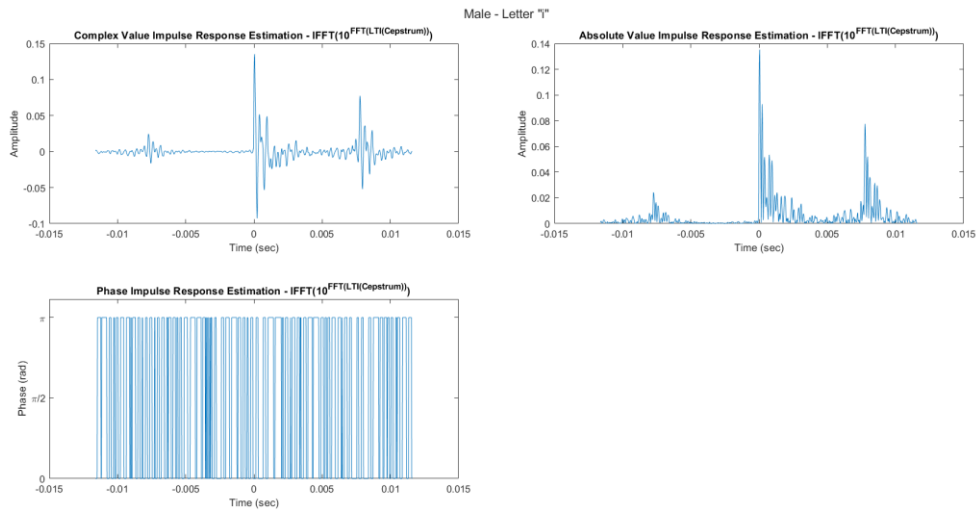
Διάγραμμα 21: Ανδρική Φωνή – Γράμμα “α” – Impulse Response



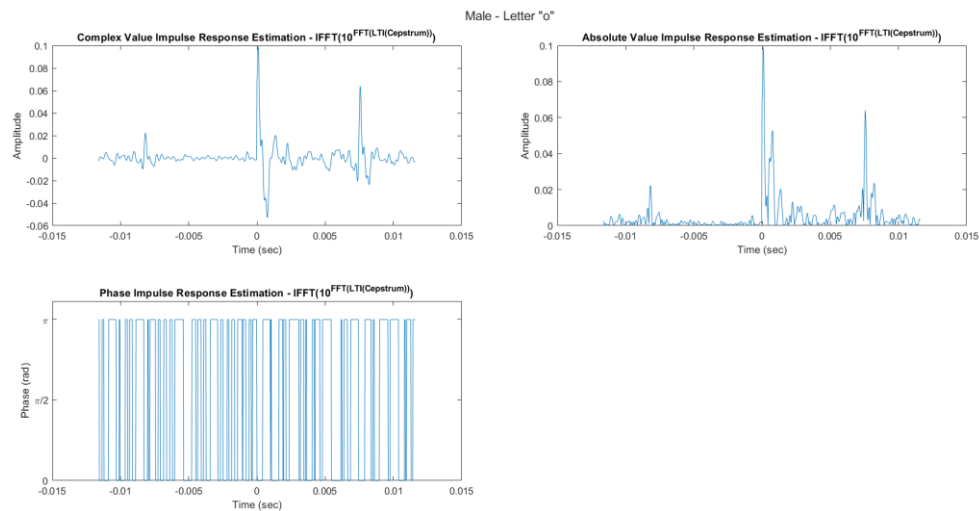
Διάγραμμα 22: Ανδρική Φωνή – Γράμμα “e” – Impulse Response



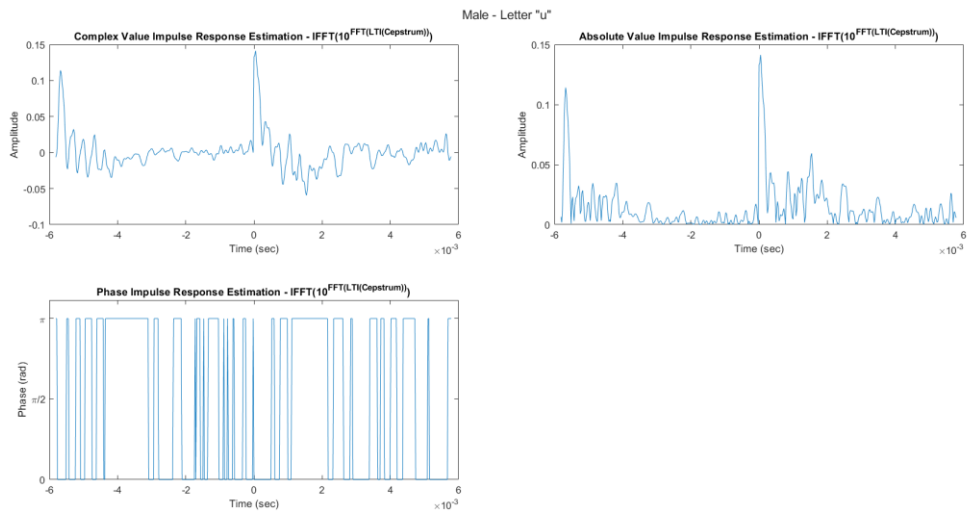
Διάγραμμα 23: Ανδρική Φωνή – Γράμμα “i” – Impulse Response



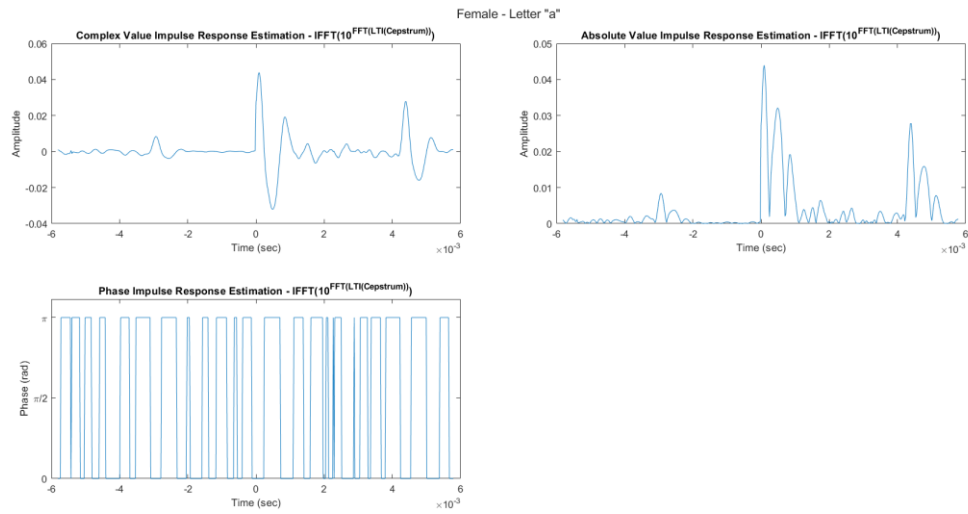
Διάγραμμα 24: Ανδρική Φωνή – Γράμμα “o” – Impulse Response



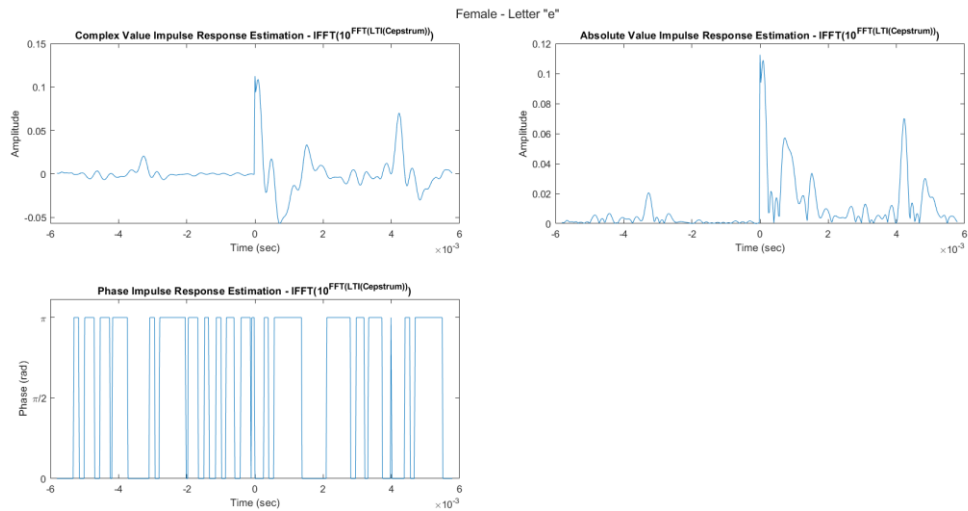
Διάγραμμα 25: Ανδρική Φωνή – Γράμμα “υ” – Impulse Response



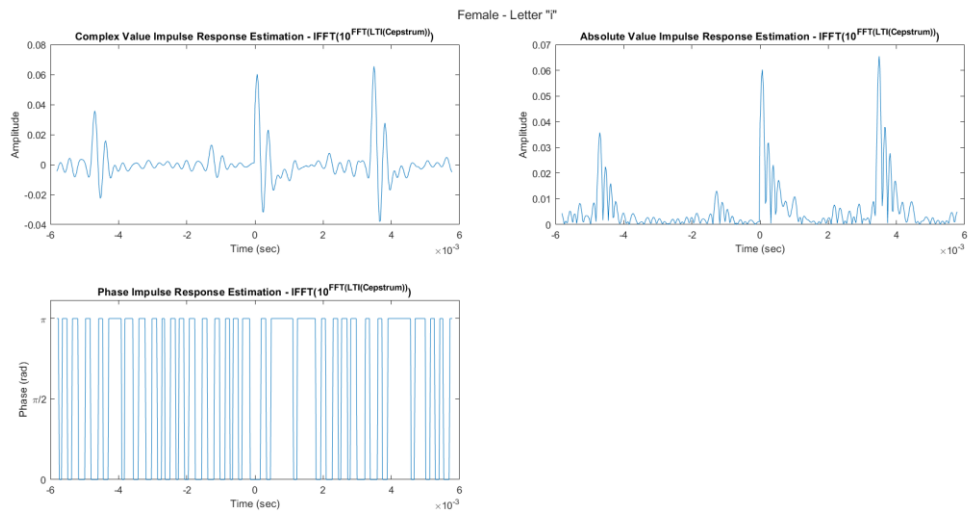
Διάγραμμα 26: Γυναικεία Φωνή – Γράμμα “α” – Impulse Response



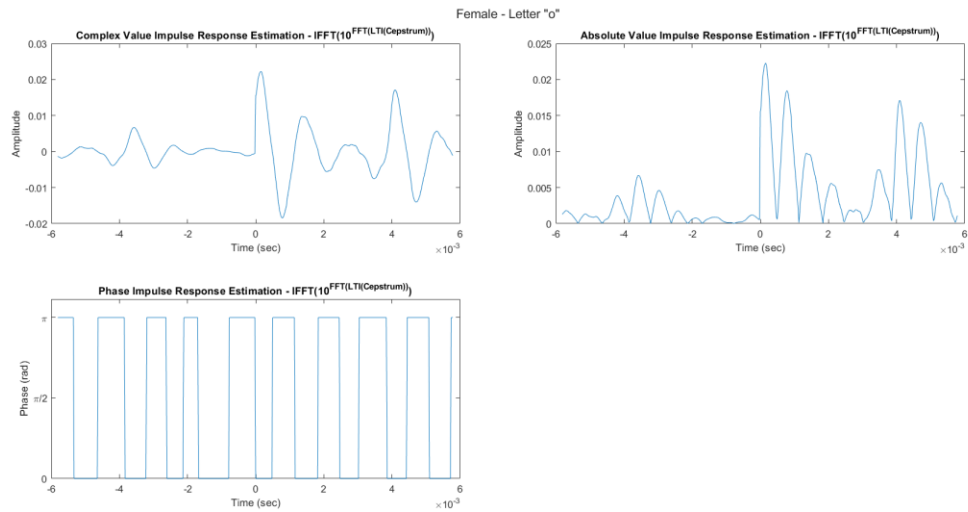
Διάγραμμα 27: Γυναικεία Φωνή – Γράμμα “ε” – Impulse Response



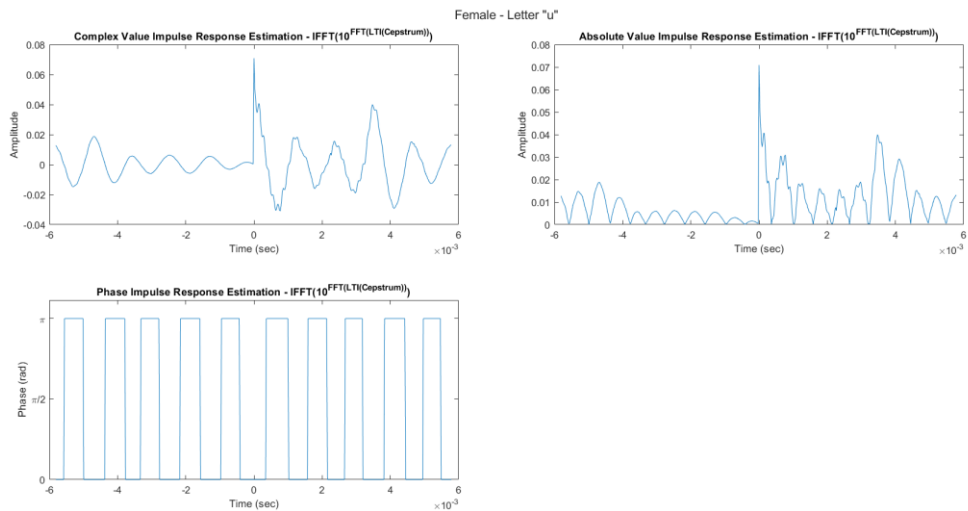
Διάγραμμα 28: Γυναικεία Φωνή – Γράμμα “ι” – Impulse Response



Διάγραμμα 29: Γυναικεία Φωνή – Γράμμα “ο” – Impulse Response



Διάγραμμα 30: Γυναικεία Φωνή – Γράμμα “υ” – Impulse Response



Τα σήματα που προκύπτουν στο πεδίο του χρόνου δεν είναι συμμετρικά ως προς την αρχή των αξόνων. Παρατηρείται τοπικό μέγιστο και η δεύτερη μεγαλύτερη κορύφωση της κυματομορφής στο χρονικό σημείο το οποίο αντιστοιχεί με τη θεμελιώδη περίοδο του σήματος.

4^ο Στάδιο: Στο στάδιο αυτό ζητείται η εκτίμηση του αρχικού-πρωτότυπου σήματος μέσω της συνέλιξης της εκτίμησης της κρουστικής απόκρισης (*impulse response*) με την εκτίμηση του παραγόμενου τόνου από τις φωνητικές χορδές (*pitch*). Για να γίνει αυτό πρέπει πρώτα να γίνει η εκτίμηση του *pitch*, η οποία γίνεται μέσω του πεδίου της συχνότητας με χρήση του *fft* και είναι μία διαδικασία που ονομάζεται *deconvolution*. Ουσιαστικά, αφού είναι γνωστό το αρχικό σήμα και η εκτίμηση της κρουστικής απόκρισης γίνεται η εκτίμηση του σήματος *pitch* παίρνοντας τον *ifft* του λόγου του *fft* του πρωτότυπου σήματος προς τον *fft* της εκτίμησης της κρουστικής απόκρισης.

Το στάδιο αυτό έχει υλοποιηθεί σε θεωρητικό επίπεδο στα πλαίσια συνάρτησης αλλά δεν έχει συνδεθεί με τον υπόλοιπο κώδικα ώστε να παρουσιαστούν αποτελέσματα.

Για να γίνει εν τέλη η σύνθεση και συνεπώς εκτίμηση του αρχικού φωνητικού σήματος γίνεται συνέλιξη μεταξύ της εκτίμησης της κρουστικής απόκρισης και της εκτίμησης του *pitch*.

Κώδικας 'MATLAB'

Ο κώδικας που αναπτύχθηκε αποτελείται από τέσσερις κύριες συναρτήσεις, μία για το κάθε στάδιο της εργασίας, μερικές βοηθητικές συναρτήσεις και μία συνάρτηση επίδειξης. Συγκεκριμένα, η συνάρτηση *'recording.m'* αφορά το πρώτο στάδιο της εργασίας, η *'calculateCepstrumPreProcessing.m'* το δεύτερο στάδιο, η *'calculateInverseCepstrum.m'* το τρίτο και η *'estimatePitch.m'* το τέταρτο. Ως βοηθητική, έχει υλοποιηθεί η *'phaseReduction.m'* για την απεικόνιση της φάσης, με στόχο να την ανάγει στο διάστημα $[-\pi, \pi]$ και τέλος η *'demo.m'* που παρουσιάζει την λειτουργικότητα του προγράμματος. Ο κώδικας των συναρτήσεων παρατίθεται παρακάτω.

Η *'recording.m'* είναι περισσότερο ένα δοκιμαστικό αρχείο και όχι συνάρτηση, καθώς τα φωνητικά δείγματα ηχογραφήθηκαν ένα προς ένα και η αποθήκευσή τους έγινε εκ των υστέρων.

Η *'estimatePitch.m'*, όπως έχει ήδη αναφερθεί, δεν χρησιμοποιείται στον κώδικα, είναι μία θεωρητική υλοποίηση της εκτίμησης του *pitch*.

```
'recording.m'

samplingRate = 44100;
bitsPerSample = 16;
numberOfChannels = 1;
recordingDuration = 3;
recObj = audiorecorder(samplingRate,bitsPerSample,numberOfChannels);
recordblocking(recObj,recordingDuration);
audioData = getaudiodata(recObj);
play(recObj)
plot(audioData)
```

```
'calculateCepstrumPreProcessing.m', (1)
function [cepstrum,fundamental_period,chosen_signal] =
calculateCepstrumPreProcessing(ogsig,samplingRate,noperiods_winlen,gender,
label,pltspr)

    if nargin==5
        pltspr=false;
    end

    prepro_ogsig = ogsig-mean(ogsig);
    prepro_ogsig = prepro_ogsig/abs(max(prepro_ogsig));

    autocorr_prepro_ogsig =
autocorr(prepro_ogsig,'NumLags',length(prepro_ogsig)-1);
    [autocorr_pks,autocorr_pks_locs] = findpeaks(autocorr_prepro_ogsig);
    [~,ind_maxpeak] = max(autocorr_pks);
    fundamental_period = autocorr_pks_locs(ind_maxpeak);
    fundamental_period = fundamental_period-1;
    fundamental_period_sec = fundamental_period/samplingRate;
    fundamental_frequency_hz = 1/fundamental_period_sec;

    [~,window_center] = max(prepro_ogsig);
    window_length = round(noperiods_winlen*fundamental_period);
```

```

        'calculateCepstrumPreProcessing.m', (2)
if mod(window_length,2)==0
    window_start = window_center-window_length/2+1;
    window_end = window_center+window_length/2;
else
    window_start = window_center-floor(window_length/2);
    window_end = window_center+floor(window_length/2);
end
window = hamming(window_length);

chosen_signal = prepro_ogsig(window_start:window_end);
windowed_signal = chosen_signal.*window;

nozerospadded = 2^nextpow2(length(windowed_signal))-
length(windowed_signal);
ppovs = vertcat(windowed_signal,zeros(nozerospadded,1));

ppovs_fft = fft(ppovs);
ppovs_fft = fftshift(ppovs_fft);
ppovs_log_fft = log10(abs(ppovs_fft));
cepstrum = ifft(ppovs_log_fft);
cepstrum = ifftshift(cepstrum);

freq_axis = (-length(ppovs_fft)/2:length(ppovs_fft)/2-
1)*samplingRate/length(ppovs_fft);
time_axis_prepro_ogsig = (0:length(ogsig)-1)*(1/samplingRate);
time_axis_cep = (-length(ppovs_fft)/2:length(ppovs_fft)/2-
1)*1/samplingRate;
time_axis_window = (window_start:window_end)*1/samplingRate;
time_axis_ppovs =
(window_start:window_end+nozerospadded)*1/samplingRate;

switch pltspr
case true
    figure()
    nexttile()
    plot(time_axis_prepro_ogsig,prepro_ogsig)
    ylabel('Amplitude')
    xlabel('Time (sec)')
    title('Original Voice Signal')
    subtitle('Normalized & Mean Extracted')

    nexttile()
    plot(time_axis_window,chosen_signal)
    ylabel('Amplitude')
    xlabel('Time (sec)')
    title('Chosen Part Of The Signal')
    subtitle('Fundamental Period:
'+string(fundamental_period_sec)+'sec')

    nexttile()
    plot(time_axis_window,hamming(window_length))
    ylabel('Amplitude')
    xlabel('Time (sec)')
    title('Hamming Window')

    nexttile()
    plot(time_axis_window>windowed_signal)
    ylabel('Amplitude')
    xlabel('Time (sec)')
    title('Windowed Signal')

    nexttile()
    plot(time_axis_ppovs,ppovs)
    ylabel('Amplitude')
    xlabel('Time (sec)')
    title('Pre-Processed Original Voice Signal (PPOVS)')
    subtitle('Zero-Padded Windowed Signal')

```

```

        'calculateCepstrumPreProcessing.m', (3)
sgttitle(gender+' - Letter "' + label + '"')

figure()
nexttile()
hold on
xline(0)
yline(0)
plot(ppovs_fft, 'o')
hold off
ylabel('Imaginary (Im)')
xlabel('Real (Re)')
title('Complex Points Distribution - FFT(PPOVS)')

nexttile()
plot(freq_axis, abs(ppovs_fft))
ylabel('Amplitude')
xlabel('Frequency (Hz)')
title('Absolute Value - |FFT(PPOVS)|')

nexttile()
plot(freq_axis, phaseReduction(phase(ppovs_fft)))
yticks([-pi, -pi/2, 0, pi/2, pi])
yticklabels({'-\pi', '-\pi/2', '0', '\pi/2', '\pi'})
ylabel('Phase (rad)')
xlabel('Frequency (Hz)')
title('Phase - FFT(PPOVS)')

nexttile()
plot(freq_axis, ppovs_log_fft)
ylabel('Amplitude (dB)')
xlabel('Frequency (Hz)')
title('Common Logarithm - Log_{10}(|FFT(PPOVS)|)')

sgttitle(gender+' - Letter "' + label + '"')

figure()
nexttile()
plot(time_axis_cep, cepstrum)
ylabel('Amplitude')
xlabel('Quefrency (sec)')
title('Complex Cepstrum - IFFT(Log_{10}(|FFT(PPOVS)|))')

nexttile()
plot(time_axis_cep, abs(cepstrum))
ylabel('Amplitude')
xlabel('Quefrency (sec)')
title('Absolute Value Of Cepstrum -
|IFFT(Log_{10}(|FFT(PPOVS)|))|')

nexttile()
plot(time_axis_cep, phaseReduction(phase(cepstrum)))
yticks([-pi, -pi/2, 0, pi/2, pi])
yticklabels({'-\pi', '-\pi/2', '0', '\pi/2', '\pi'})
ylabel('Phase (rad)')
xlabel('Quefrency (sec)')
title('Phase Of Cepstrum - IFFT(Log_{10}(|FFT(PPOVS)|))')

sgttitle(gender+' - Letter "' + label + '"')

end
end

```

```

        'calculateInverseCepstrum.m', (1)
function impressp_estim =
calculateInverseCepstrum(cepstrum,fundamental_period,noperiods_winlen,samp
lingRate,gender,label,pltspr)

    if nargin==5
        pltspr = false;
    end

    window_length = round(noperiods_winlen*fundamental_period);
    window_center = floor(length(cepstrum)/2)+1;
    if mod(window_length,2)==0
        window_start = window_center-floor(window_length/2)+1;
        window_end = window_center+floor(window_length/2);
    else
        window_start = window_center-floor(window_length/2);
        window_end = window_center+floor(window_length/2);
    end
    window = zeros(length(cepstrum),1);
    window(window_start:window_end) = rectwin(window_length);

    liftered_cepstrum = cepstrum.*window;
    liftered_cepstrum_cut = liftered_cepstrum(liftered_cepstrum~=0);

    signal_to_fft =
    liftered_cepstrum(floor(length(liftered_cepstrum)/2)+1:end);
    nozerospadded = 2^nextpow2(length(signal_to_fft))-
length(signal_to_fft);
    signal_to_fft = vertcat(signal_to_fft,zeros(nozerospadded,1));
    liftered_cepstrum_fft = fft(signal_to_fft);
    liftered_cepstrum_fft = fftshift(liftered_cepstrum_fft);
    liftered_cepstrum_exp_fft = power(10,liftered_cepstrum_fft);

    impressp_estim = ifft(liftered_cepstrum_exp_fft);
    impressp_estim = ifftshift(impressp_estim);

    freq_axis_liftcep = (-
length(liftered_cepstrum_fft)/2:length(liftered_cepstrum_fft)/2-
1)*samplingRate/length(liftered_cepstrum_fft);
    time_axis_liftcep = (-
length(liftered_cepstrum)/2:length(liftered_cepstrum)/2-1)*1/samplingRate;
    time_axis_liftcep_cut = (-
length(liftered_cepstrum_cut)/2:length(liftered_cepstrum_cut)/2-
1)*1/samplingRate;
    time_axis_impressp_estim = (-
length(impressp_estim)/2:length(impressp_estim)/2-1)*1/samplingRate;

    switch pltspr
        case true
            figure()
            nexttile()
            plot(time_axis_liftcep_cut,window(window_start:window_end))
            ylabel('Amplitude')
            xlabel('Quefreny (sec)')
            title('Rectangular Window')
            subtitle('Low Pass Filter - LTI')

            nexttile()
            plot(time_axis_liftcep,cepstrum)
            ylabel('Amplitude')
            xlabel('Quefreny (sec)')
            title('Complex Cepstrum - IFFT(Log_{10}(|FFT(PPOVS)|))')

            nexttile()
            plot(time_axis_liftcep,liftered_cepstrum)
            ylabel('Amplitude')
            xlabel('Quefreny (sec)')
            title('Liftered Complex Cepstrum - LTI(Cepstrum)')

```

```

        'calculateInverseCepstrum.m', (2)
nexttile()
plot(time_axis_liftcep_cut,liftered_cepstrum_cut)
ylabel('Amplitude')
xlabel('Quefreny (sec)')
title('Liftered Complex Cepstrum Cut - LTI(|Cepstrum|)')
subtitle('Omitting Zero Values - High Frequencies')

sgtitle(gender+' - Letter "' +label+'"');

figure()
nexttile()
hold on
yline(0)
xline(0)
plot(liftered_cepstrum_fft,'o')
hold off
ylabel('Imaginary (Im)')
xlabel('Real (Re)')
title('Complex Points Distribution - FFT(LTI(Cepstrum))')

nexttile()
plot(freq_axis_liftcep,abs(liftered_cepstrum_fft))
ylabel('Amplitude (dB)')
xlabel('Frequency (Hz)')
title('Absolute Value FFT(LTI(Cepstrum))')

nexttile()

plot(freq_axis_liftcep,phaseReduction(phase(liftered_cepstrum_fft)))
yticks([-pi,-pi/2,0,pi/2,pi])
yticklabels({'-\pi','-\pi/2','0','\pi/2','\pi'})
ylabel('Phase (rad)')
xlabel('Frequency (Hz)')
title('Phase FFT(LTI(Cepstrum))')

nexttile()
hold on
yline(0)
xline(0)
plot(liftered_cepstrum_exp_fft,'o')
hold off
ylabel('Imaginary (Im)')
xlabel('Real (Re)')
title('Complex Points Distribution - 10^{FFT(LTI(Cepstrum))}')

nexttile()
plot(freq_axis_liftcep,abs(liftered_cepstrum_exp_fft))
ylabel('Amplitude')
xlabel('Frequency (Hz)')
title('Absolute Value 10^{FFT(LTI(Cepstrum))}')

nexttile()

plot(freq_axis_liftcep,phaseReduction(phase(liftered_cepstrum_exp_fft)))
yticks([-pi,-pi/2,0,pi/2,pi])
yticklabels({'-\pi','-\pi/2','0','\pi/2','\pi'})
ylabel('Phase (rad)')
xlabel('Frequency (Hz)')
title('Phase 10^{FFT(LTI(Cepstrum))}')

sgtitle(gender+' - Letter "' +label+'"');

```

```

        'calculateInverseCepstrum.m', (3)
        figure()
        nexttile()
        plot(time_axis_impresp_estim,impresp_estim)
        ylabel('Amplitude')
        xlabel('Time (sec)')
        title('Complex Value Impulse Response Estimation -
IFFFT(10^{FFT(LTI(Cepstrum))})')

        nexttile()
        plot(time_axis_impresp_estim,abs(impresp_estim))
        ylabel('Amplitude')
        xlabel('Time (sec)')
        title('Absolute Value Impulse Response Estimation -
IFFFT(10^{FFT(LTI(Cepstrum))})')

        nexttile()

        plot(time_axis_impresp_estim,phaseReduction(phase(impresp_estim)))
        yticks([-pi,-pi/2,0,pi/2,pi])
        yticklabels({'-\pi','-\pi/2','0','\pi/2','\pi'})
        ylabel('Phase (rad)')
        xlabel('Time (sec)')
        title('Phase Impulse Response Estimation -
IFFFT(10^{FFT(LTI(Cepstrum))})')

        sgtitle(gender+' - Letter "' +label+'"');
    end
end

```

```

        'estimatePitch.m'
function pitch_estim = estimatePitch(impresp,voicesig)

    impresp = vertcat(impresp,zeros(length(impresp),1));

    voicesig_fft = fft(voicesig);
    voicesig_fft = fftshift(voicesig_fft);

    impresp_fft = fft(impresp);
    impresp_fft = fftshift(impresp_fft);

    faketrans_function = voicesig_fft./impresp_fft;

    pitch_estim = ifft(faketrans_function);
    pitch_estim = ifftshift(pitch_estim);

end

```

```

        'phaseReduction.m'
function phasered = phaseReduction(phase)

    phasered = mod(phase,2*pi);

    pos = find(phase>=0);
    neg = find(phase<0);
    pos_pito2pi = find(phasered(pos)>pi);
    neg_pito2pi = find(phasered(neg)>pi);

    phasered(pos(pos_pito2pi)) = phasered(pos(pos_pito2pi))-2*pi;
    phasered(neg(neg_pito2pi)) = phasered(neg(neg_pito2pi))-2*pi;

end

```

```

                                'demo.m'

clf
close all
clear
clc

load('male_voice.mat')
load('female_voice.mat')

male = {};
female = {};

male_window_coeffs = [3.3;4.1;4;4.3;3.7];
female_window_coeffs = [3.4;3.55;4.65;3.89;4.84];
male_filter_coeffs = [3;3;3;3;3];
female_filter_coeffs = [3;3;3;3;3];

for i=1:length(male_voice)
    [cepstrum,fundamental_period,chosen_signal] =
    calculateCepstrumPreProcessing(male_voice{i}.Audio,male_voice{i}.SamplingR
ate,male_window_coeffs(i),"Male",male_voice{i}.Label,true);
    impulse_response_estimation =
    calculateInverseCepstrum(cepstrum,fundamental_period,male_filter_coeffs(i)
,male_voice{i}.SamplingRate,"Male",male_voice{i}.Label,true);

    male{i} =
    struct('Label',male_voice{i}.Label,'Signal',chosen_signal,'FundamentalPeri
od',fundamental_period/male_voice{i}.SamplingRate,'Cepstrum',cepstrum,'Im
pulseResponse',impulse_response_estimation);
end

for i=1:length(female_voice)
    [cepstrum,fundamental_period,chosen_signal] =
    calculateCepstrumPreProcessing(female_voice{i}.Audio,female_voice{i}.Sampl
ingRate,female_window_coeffs(i),"Female",female_voice{i}.Label,true);
    impulse_response_estimation =
    calculateInverseCepstrum(cepstrum,fundamental_period,female_filter_coeffs(
i),female_voice{i}.SamplingRate,"Female",female_voice{i}.Label,true);

    female{i} =
    struct('Label',female_voice{i}.Label,'Signal',chosen_signal,'FundamentalPe
riod',fundamental_period/female_voice{i}.SamplingRate,'Cepstrum',cepstrum
,'ImpulseResponse',impulse_response_estimation);
end

clear cepstrum chosen_signal fundamental_period i
impulse_response_estimation window

```